SIMULATION EXPERIMENT – 2

FORMAL DESIGN OF AN EMBEDDED CONTROLLER USING STATE MACHINE (STATECHART) MODELING – TRAFFIC LIGHT CONTROLLER

The format and section structure below follow the uploaded manual, including Objectives, Expected Outcome, Software/Hardware, Theory, Procedure, Program, Parameter Tables, Algorithm, Key Conditions, Rubric, Sample Lab Questions, and Result. Pasted text

➔ Objectives of this Experiment:

1. To understand the concept of Finite State Machine (FSM) and Statechart modeling for embedded controller design.
2. To formally represent the operation of a traffic light controller using defined states, transitions, timing conditions, and outputs.
3. To implement a four-state traffic light controller using an Arduino UNO.
4. To simulate the traffic light controller using Proteus Design Suite.
5. To verify the state transitions and corresponding traffic light outputs using the Proteus Virtual Terminal.

➔ Expected Outcome from this Experiment:

Students shall be able to:

1. Develop a Statechart/FSM model for a simple embedded control application.

2. Identify states, transitions, timing conditions, and output actions in an embedded controller.
3. Implement an FSM-based traffic light controller using Embedded C.
4. Configure Arduino UNO digital output pins for controlling traffic signal LEDs.
5. Simulate and verify the FSM operation using Proteus software and a Virtual Terminal.

➔ Software Required:

S. No.	Software	Purpose
1	Proteus Design Suite 8	Circuit design and simulation
2	Arduino IDE	Embedded C program development and HEX file generation

➔ Hardware Components Used in Simulation:

S. No.	Component	Quantity
1	Arduino UNO	1
2	Red LED	2
3	Yellow LED	2
4	Green LED	2
5	220 Ω Resistor	6

6	**Virtual Terminal**	**1**
7	**Ground**	**As required**

➔ Objective Linkage:

The experiment enables students to:

- Model an embedded controller using a finite state machine.
- Define discrete operating states for a traffic signal controller.
- Define state transitions based on timing conditions.
- Assign appropriate output actions to each state.
- Implement the Statechart model using Embedded C.
- Verify the correspondence between FSM states and physical LED outputs.
- Monitor state transitions using the Proteus Virtual Terminal.

➔ Theory:

A Finite State Machine (FSM) is a formal computational model used to describe the behavior of an embedded system through a finite number of well-defined states and transitions.

An FSM generally consists of:

- States
- Inputs/events or conditions
- Transitions
- Outputs/actions

In this experiment, a traffic light controller is represented using four states. Each state defines the condition of the North-South and East-West traffic signals.

State S0 – North-South Green

North-South → GREEN

East-West → RED

Duration → 10 seconds

After 10 seconds, the controller transitions to S1.

State S1 – North-South Yellow

North-South → YELLOW

East-West → RED

Duration → 3 seconds

After 3 seconds, the controller transitions to S2.

State S2 – East-West Green

North-South → RED

East-West → GREEN

Duration → 10 seconds

After 10 seconds, the controller transitions to S3.

State S3 – East-West Yellow

North-South → RED

East-West → YELLOW

Duration → 3 seconds

After 3 seconds, the controller returns to S0.

Therefore, the complete state sequence is:

S0 → S1 → S2 → S3 → S0 → ...

The Statechart can be represented as:

10 s

➔ STATE TRANSITION TABLE:

Present State	State Description	Condition	Next State
S0	**NS Green / EW Red**	**10 s elapsed**	**S1**
S1	**NS Yellow / EW Red**	**3 s elapsed**	**S2**
S2	**NS Red / EW Green**	**10 s elapsed**	**S3**
S3	**NS Red / EW Yellow**	**3 s elapsed**	**S0**

➔ OUTPUT STATE TABLE:

State	NS Red	NS Yellow	NS Green	EW Red	EW Yellow	EW Green
S0	**0**	**0**	**1**	**1**	**0**	**0**
S1	**0**	**1**	**0**	**1**	**0**	**0**
S2	**1**	**0**	**0**	**0**	**0**	**1**
S3	**1**	**0**	**0**	**0**	**1**	**0**

➔ Arduino PIN ASSIGNMENT:

Arduino Pin	Signal	Function

D2	**NS Red**	**North-South Red LED**
D3	**NS Yellow**	**North-South Yellow LED**
D4	**NS Green**	**North-South Green LED**
D5	**EW Red**	**East-West Red LED**
D6	**EW Yellow**	**East-West Yellow LED**
D7	**EW Green**	**East-West Green LED**
D1 / TX	**Serial Output**	**Virtual Terminal RXD**

Each LED is connected through a 220 Ω current-limiting resistor.

➔ Experimental Procedure:

A. Statechart and FSM Development

1. Identify the traffic signal control problem.
2. Define the four operating states of the controller.
3. Assign appropriate signal outputs to each state.
4. Define the timing condition associated with each state.
5. Develop the Statechart showing the state transitions.
6. Prepare the State Transition Table.
7. Prepare the Output State Table.

B. Proteus Circuit Development

1. Open the Proteus Design Suite.
2. Create a new schematic design.
3. Select and place the Arduino UNO.

4. Place two red LEDs, two yellow LEDs, and two green LEDs.
5. Place six 220 Ω resistors.
6. Connect the LEDs to Arduino digital pins D2–D7.
7. Connect the LED returns to GND.
8. Place a Virtual Terminal.
9. Connect Arduino D1/TX to the Virtual Terminal RXD.
10. Leave TXD, RTS, and CTS of the Virtual Terminal unconnected.
11. Set the Virtual Terminal communication speed to 9600 baud.
12. Complete all circuit connections.

C. Program Development and HEX Generation

1. Open Arduino IDE.
2. Select Arduino UNO as the target board.
3. Develop the Embedded C program based on the FSM.
4. Compile the program.
5. Verify that there are no compilation errors.
6. Select Sketch → Export Compiled Binary.
7. Generate the .hex file.
8. Load the generated HEX file into the Arduino UNO component in Proteus.

D. Program Execution and Verification

1. Start the Proteus simulation.

2. Observe the initial state S0.
3. Verify that North-South Green and East-West Red are active.
4. Observe the Virtual Terminal.
5. Verify the message:

STATE S0: NS GREEN - EW RED

6. After 10 seconds, verify transition to S1.
7. Verify North-South Yellow and East-West Red.
8. After 3 seconds, verify transition to S2.
9. Verify North-South Red and East-West Green.
10. After 10 seconds, verify transition to S3.
11. Verify North-South Red and East-West Yellow.
12. After 3 seconds, verify return to S0.
13. Verify that the complete state sequence repeats continuously.

➔ PROGRAM:

```
// ------------------------------------------------
// Formal Design of Embedded Controller Using FSM
// Traffic Light Controller
// Arduino UNO + Proteus
// ------------------------------------------------


enum TrafficState
```

```
{
  STATE_NS_GREEN,
  STATE_NS_YELLOW,
  STATE_EW_GREEN,
  STATE_EW_YELLOW
};

TrafficState currentState = STATE_NS_GREEN;

// North-South traffic signal
const int NS_RED_PIN    = 2;
const int NS_YELLOW_PIN = 3;
const int NS_GREEN_PIN  = 4;

// East-West traffic signal
const int EW_RED_PIN    = 5;
const int EW_YELLOW_PIN = 6;
const int EW_GREEN_PIN  = 7;

void setup()
{
  pinMode(NS_RED_PIN, OUTPUT);
  pinMode(NS_YELLOW_PIN, OUTPUT);
```

```
  pinMode(NS_GREEN_PIN, OUTPUT);

  pinMode(EW_RED_PIN, OUTPUT);
  pinMode(EW_YELLOW_PIN, OUTPUT);
  pinMode(EW_GREEN_PIN, OUTPUT);

  Serial.begin(9600);
}

void allOff()
{
  digitalWrite(NS_RED_PIN, LOW);
  digitalWrite(NS_YELLOW_PIN, LOW);
  digitalWrite(NS_GREEN_PIN, LOW);

  digitalWrite(EW_RED_PIN, LOW);
  digitalWrite(EW_YELLOW_PIN, LOW);
  digitalWrite(EW_GREEN_PIN, LOW);
}

void enterState(TrafficState state)
{
  allOff();
```

```
switch (state)
{
  case STATE_NS_GREEN:

    digitalWrite(NS_GREEN_PIN, HIGH);
    digitalWrite(EW_RED_PIN, HIGH);

    Serial.println("STATE S0: NS GREEN - EW RED");

    break;

  case STATE_NS_YELLOW:

    digitalWrite(NS_YELLOW_PIN, HIGH);
    digitalWrite(EW_RED_PIN, HIGH);

    Serial.println("STATE S1: NS YELLOW - EW RED");

    break;

  case STATE_EW_GREEN:
```

```
      digitalWrite(NS_RED_PIN, HIGH);
      digitalWrite(EW_GREEN_PIN, HIGH);

      Serial.println("STATE S2: NS RED - EW GREEN");

      break;

    case STATE_EW_YELLOW:

      digitalWrite(NS_RED_PIN, HIGH);
      digitalWrite(EW_YELLOW_PIN, HIGH);

      Serial.println("STATE S3: NS RED - EW YELLOW");

      break;
  }
}

void loop()
{
  switch (currentState)
  {
    case STATE_NS_GREEN:
```

```
  enterState(STATE_NS_GREEN);
  delay(10000);
  currentState = STATE_NS_YELLOW;

  break;

case STATE_NS_YELLOW:

  enterState(STATE_NS_YELLOW);
  delay(3000);
  currentState = STATE_EW_GREEN;

  break;

case STATE_EW_GREEN:

  enterState(STATE_EW_GREEN);
  delay(10000);
  currentState = STATE_EW_YELLOW;

  break;
```

```
    case STATE_EW_YELLOW:

      enterState(STATE_EW_YELLOW);
      delay(3000);
      currentState = STATE_NS_GREEN;

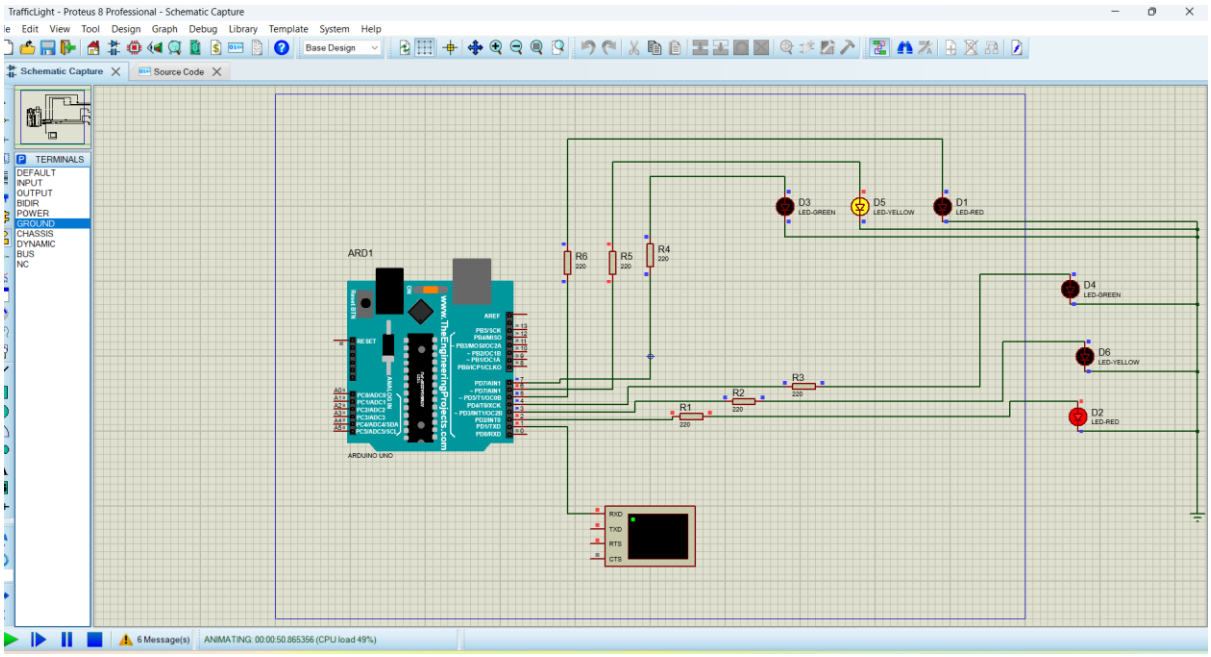
      break;
  }
}
```

➔ **Program Explanation:**

Program Element	Function
enum TrafficState	**Defines the finite set of controller states**
STATE_NS_GREEN	**Represents S0 state**
STATE_NS_YELLOW	**Represents S1 state**
STATE_EW_GREEN	**Represents S2 state**
STATE_EW_YELLOW	**Represents S3 state**
currentState	**Stores the currently active FSM state**
pinMode()	**Configures LED pins as digital outputs**
Serial.begin(9600)	**Initializes serial communication with Virtual Terminal**

allOff()	**Turns OFF all traffic LEDs before entering a new state**
enterState()	**Generates the output actions associated with each state**
switch()	**Selects the appropriate state operation**
delay(10000)	**Maintains a green state for 10 seconds**
delay(3000)	**Maintains a yellow state for 3 seconds**
currentState = ...	**Performs the state transition**
Serial.println()	**Displays the active FSM state in Virtual Terminal**

➔ CIRCUIT DIAGRAM:



Arduino UNO – Traffic Light Controller Interface

North-South signals:

D2 ── 220Ω ── RED LED ── GND

D3 ── 220Ω ── YELLOW LED ── GND

D4 ── 220Ω ── GREEN LED ── GND

East-West signals:

D5 ── 220Ω ── RED LED ── GND

D6 ── 220Ω ── YELLOW LED ── GND

D7 ── 220Ω ── GREEN LED ── GND

State monitoring:

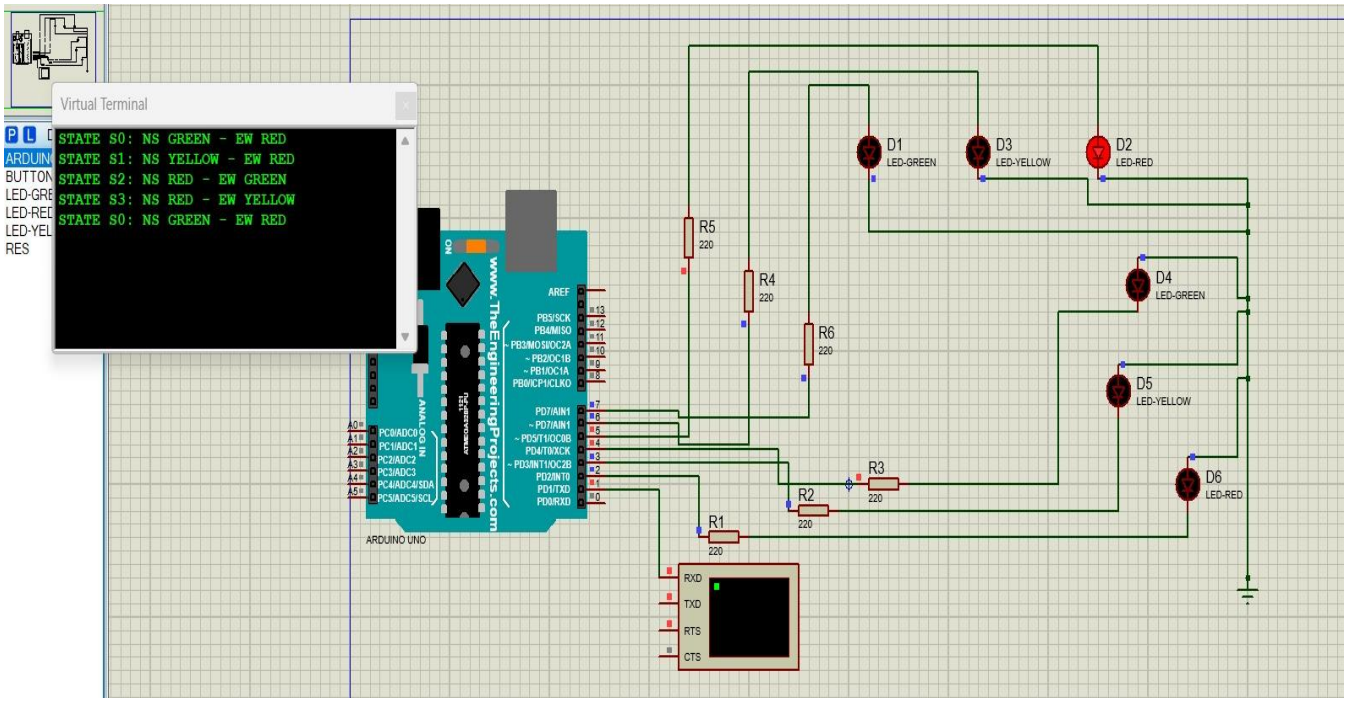
Arduino D1/TX ──────────── Virtual Terminal RXD

➔ INPUT PARAMETER TABLE – VALUES TO BE ENTERED

S. No.	Input Parameter	Symbol / Setting	Value to Enter	Unit
1	Microcontroller	Arduino UNO	Arduino UNO	—
2	NS Red Output	D2	2	Pin
3	NS Yellow Output	D3	3	Pin
4	NS Green Output	D4	4	Pin
5	EW Red Output	D5	5	Pin
6	EW Yellow Output	D6	6	Pin
7	EW Green Output	D7	7	Pin
8	NS Green Duration	TNS-G	10	s

9	**NS Yellow Duration**	**TNS-Y**	**3**	**s**
10	**EW Green Duration**	**TEW-G**	**10**	**s**
11	**EW Yellow Duration**	**TEW-Y**	**3**	**s**
12	**Serial Communication**	**Baud Rate**	**9600**	**baud**
13	**LED Resistor**	**R**	**220**	**Ω**
14	**Simulation Software**	**—**	**Proteus 8**	**—**

➔ OUTPUT:


Virtual Terminal
STATE S0: NS GREEN - EW RED
STATE S1: NS YELLOW - EW RED
STATE S2: NS RED - EW GREEN
STATE S3: NS RED - EW YELLOW
STATE S0: NS GREEN - EW RED
ARDUIN
BUTTON
LED-GRE
LED-RED
LED-YEL
RES
D1
LED-GREEN
D3
LED-YELLOW
D2
LED-RED
D4
LED-GREEN
D5
LED-YELLOW
D6
LED-RED
R1
220
R2
220
R3
220
R4
220
R5
220
R6
220
www.TheEngineeringProjects.com
ARDUINO UNO
RXD
TXD
RTS
CTS